

ඇල්ගොරිතම පරීක්ෂා කිරීම (Algorithm Testing)

Author: Yasithzz | 2026

02 -- ඇල්ගොරිතම පරීක්ෂා කිරීම හා API සමීක්ෂණය (Algorithm Testing & Optimization)

LifeMate වියාපෘතිය -- සිංහල ලේඛනය

1. app.py කුමක් කරනවාද?

ml-service/app.py ගොනුව ශික්ෂිත model (train.py විසින් සෑදූ stress_model.pkl) API ලෙස ලෝකයට ඉදිරිපත් කරයි. ලිහිල් භාෂාවෙන් -- model ගබඩා (warehouse) ලෙස සිතන්න. train.py ගබඩාවට භාණ්ඩ (model) ලබා දෙයි; app.py ඒ ගබඩාවේ ප්රවේශාසන (API endpoint) ලෙස ක්රියා කරයි -- frontend/backend software ඉල්ලුම් වට prediction ලබා දෙයි.

FastAPI framework -- Python ශ්රේණිමය API framework -- IfFire API ලෙස StructuredData.

2. app.py -- රේඛා රේඛා පැහැදිලි කිරීම

2.1 -- ශීර්ෂය (Docstring)

```
"""
LifeMate ML Service -- FastAPI
Endpoints:
  POST /predict -- predict stress level from lifestyle data
  POST /add-data -- append a labelled row to the dataset
  POST /retrain -- retrain model from updated dataset
  GET /health -- liveness probe
  GET /model-info -- returns current model accuracy + feature list
"""
```

සිංහල: app.py ගොනුවේ API endpoints 5 ක් ඇත. ලෝකය ඒ endpoints හරහා ML සමීක්ෂණය භාවිතා කරයි.

2.2 -- Imports

```
import os, json, csv, threading
import numpy as np
import pandas as pd
import joblib
from fastapi import FastAPI, HTTPException, BackgroundTasks
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field, validator
from typing import Optional
```

සිංහල:

Import	පිරිසෝජනය
`threading`	Model lock -- එකවිටම ලොකු requests 2ක් crash නොවී handle කිරීම
`joblib`	stress_model.pkl ගොනුව memory ලෙස load කිරීම
`fastapi`	API server සිදීම
`CORSMiddleware`	Frontend (React) ML service ලෙස access ලබාදීම
`pydantic`	Request data validation -- data types validate කිරීම

2.3 -- Path නියතයන්

```
BASE = os.path.dirname(__file__)
MODEL_PKL = os.path.join(BASE, "model", "stress_model.pkl")
MODEL_META = os.path.join(BASE, "model", "model_meta.json")
DATASET = os.path.join(BASE, "..", "datasets", "stress_dataset.csv")
```

සිංහල: Model (.pkl), meta-data (.json), dataset (.csv) ගොනු ස්ථාන නිශ්චිත කිරීම.

2.4 -- RETRAIN_BATCH සහ thread lock

```
RETRAIN_BATCH = 1
_new_samples_count = 0
_model_lock = threading.Lock()
```

සිංහල:

- * RETRAIN_BATCH = 1 -- නව data 1 ක් ලැබෙන සෑම විටම model ස්වයංක්රීයව නැවත ශික්ෂිත කරවේ.
- * _model_lock -- එකවිටම prediction + retrain සිදු වූදීදී crash නොවී ස්ථාවරව ක්රියා කිරීමට lock.

2.5 -- FastAPI App සිදීම

```
app = FastAPI(title="LifeMate ML Service", version="1.0.0")
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"], allow_methods=["*"], allow_headers=["*"],
)
```

සිංහල: API server සැදුමයි. CORS middleware -- frontend React app ML API ලෙස requests යවන්නට ඉඩ දෙයි (browser security bypass).

2.6 -- Model Load කිරීම

```
def load_model():
    if not os.path.exists(MODEL_PKL):
        raise RuntimeError("Model not found -- run train.py first")
    return joblib.load(MODEL_PKL)

_clf = None

def get_clf():
    global _clf
    if _clf is None:
        _clf = load_model()
    return _clf
```

සිංහල:

- * load_model() -- .pkl ගොනුව ඇත් නම් memory ලෙස load කරයි. නොමැති නම් error.
- * get_clf() -- Model ගත් (cached) ලෙස ගබ්සා ගනී -- request ගොඩාකදී model file නැවත load නොකර cache memory ඡීරනය භාවිතා කරයි.

2.7 -- LifestyleInput Schema (Data Validation)

```
class LifestyleInput(BaseModel):
    mood: int = Field(..., ge=1, le=5)
    workload: int = Field(..., ge=1, le=5)
    sleep_hours: float = Field(..., ge=0, le=12)
    energy_level: int = Field(..., ge=1, le=5)
    social_interaction: int = Field(3, ge=1, le=5)
    exercise_done: int = Field(0, ge=0, le=1)
    screen_time_hours: float = Field(4.0, ge=0, le=14)
    water_cups: float = Field(6.0, ge=0, le=16)
```

සිංහල: Pydantic Schema -- Frontend ලිහිල් ලෙස යවන data validate කරයි.

Field	Sinhala	සීමා
mood	හැඟීම	1-5
workload	වැඩ බරය	1-5
sleep_hours	නිදිමත	0-12
energy_level	ශක්තිය	1-5
social_interaction	සමාජ ගනුදෙනු	1-5 (Default: 3)
exercise_done	ඒයායාම	0 හෝ 1 (Default: 0)
screen_time_hours	Screen කාලය	0-14 (Default: 4)
water_cups	ජල කෝප්ප	0-16 (Default: 6)

2.8 -- /health Endpoint

```
@app.get("/health")
def health():
    return {"status": "ok", "model_loaded": os.path.exists(MODEL_PKL)}
```

සිංහල: API server ක්‍රියාත්මකද, model file ඇතිද යන්න පරීක්ෂා කිරීමේ endpoint.

2.9 -- /model-info Endpoint

```
@app.get("/model-info")
def model_info():
    if not os.path.exists(MODEL_META):
        raise HTTPException(404, "Model metadata not found")
    with open(MODEL_META) as f:
        return json.load(f)
```

සිංහල: model_meta.json කියවා features, labels, accuracy ලබා දෙයි -- backend/admin panels යැවිය.

2.10 -- /predict Endpoint <- ප්‍රධාන Endpoint

```
@app.post("/predict", response_model=PredictResponse)
def predict(data: LifestyleInput):
    clf = get_clf()
    X = np.array([[
        data.mood, data.workload, data.sleep_hours, data.energy_level,
        data.social_interaction, data.exercise_done,
        data.screen_time_hours, data.water_cups,
    ]])
    with _model_lock:
        pred = int(clf.predict(X)[0])
        proba = clf.predict_proba(X)[0]

    return PredictResponse(
        stress_level=pred,
        stress_label=LABELS[pred],
        probabilities={LABELS[i]: round(float(p), 4) for i, p in enumerate(proba)},
    )
```

සිංහල -- කිරියාවලිය:

- * Frontend 8 features (mood, workload, ...) POST request ලෙස යවයි.
- * Pydantic validate කරයි -- data valid නොවනේ නම් 422 error.
- * get_clf() -- ගිණිතින model load.
- * np.array -- 8 features 1×8 matrix ලෙස සකස් කිරීම.
- * clf.predict(X) -- model ශීර්ෂණය ගණනය.
- * clf.predict_proba(X) -- සෑම stress ශීර්ෂණයකටම ප්‍රායෝගිකත්වය (probability) ගණනය.
- * PredictResponse -- JSON response: stress_level (number), stress_label (text), probabilities.

```
{
  "stress_level": 2,
  "stress_label": "Normal",
  "probabilities": {
    "Very Low": 0.05,
    "Low": 0.10,
    "Normal": 0.65,
    "High": 0.15,
    "Very High": 0.05
  }
}
```

2.11 -- /add-data Endpoint (Live Learning)

```
@app.post("/add-data")
def add_data(data: AddDataRequest, background_tasks: BackgroundTasks):
    row = [data.mood, data.workload, data.sleep_hours, data.energy_level,
           data.social_interaction, data.exercise_done,
           data.screen_time_hours, data.water_cups, data.stress_level]
    with open(DATASET, "a", newline="") as f:
        writer = csv.writer(f)
        writer.writerow(row)
    _new_samples_count += 1
    if _new_samples_count >= RETRAIN_BATCH:
        background_tasks.add_task(_do_retrain)
```

සිංහල:

- * නව confirmed data (stress level ද ඇතුළත්) dataset ගොනුවට ලියයි.
- * RETRAIN_BATCH (1) ලෙස නිශ්චිත -- data 1 ලිවූ සෑම විටම background retrain trigger.

2.12 -- /retrain Endpoint

```
@app.post("/retrain")
def retrain_endpoint(background_tasks: BackgroundTasks):
    background_tasks.add_task(_do_retrain)
    return {"status": "retraining started in background"}
```

සිංහල: Admin/system manual ලෙස model retrain trigger කළ හැකි endpoint.

2.13 -- _do_retrain() Background Task

```
def _do_retrain():
    global _clf
    try:
        import subprocess, sys
        result = subprocess.run(
            [sys.executable, os.path.join(BASE, "train.py")],
            capture_output=True, text=True, cwd=BASE)
        if result.returncode == 0:
            with _model_lock:
                _clf = load_model()
    except Exception as e:
        print(f"[ML] Retrain error: {e}")
```

සිංහල:

- * subprocess.run -- python train.py command background ලෙස ධාවනය.
- * ශික්ෂිත සාර්ථකව නම් -- lock ලෙස _clf ගබ්සාව නව model ලෙස update.
- * API restart නොකර live ලෙස model update -- continuous learning.

3. Prediction ලැබෙන ක්රමය -- සම්පූර්ණ flow

```
User (Frontend)
  v POST /predict (8 lifestyle values)
FastAPI app.py
  v Pydantic validate
  v np.array -- matrix සෑදීම
  v clf.predict() -- RandomForest model ශීර්ෂණය
  v clf.predict_proba() -- probabilities
  v JSON response
Frontend
  v stress_label ("Normal"), stress_level (2), probabilities
User screen -- stress result display
```

ලේඛනය -- LifeMate ව්‍යාපෘතිය, Yasithzz, 2026